TASK 10 – Compare Expectations with Execution

1. Objective

To compare the expected and actual behaviour of the metro fare system by calculating total revenue using double and integer-paise representations for 20,000, 500,000 and 3,000,000 journeys. The task also evaluates integer overflow, floating-point precision, compiler warnings and the suitability of different data types for large-scale calculations.

2. Expected Behaviour

Two approaches are used to calculate the total fare:

- Double representation: Stores fare values in rupees using floating-point numbers.
- Integer-paise representation: Stores fares in paise using long long, avoiding floating-point rounding errors.

For a fair comparison, the same five-journey fare pattern is repeated:

Journey	**Fare**
1	Rs. 10.00
2	Rs. 24.00
3	Rs. 30.00
4	Rs. 12.00

5	Rs. 20.00
Total	Rs. 96.00

Therefore:

$AverageFare = \frac{96}{5} = Rs.19.20$

3. Expected and Calculated Network Totals

Using the same five-journey pattern:

Journeys	**Double Total (Rs.)**	**Integer-Paise Total (Rs.)**	**Difference**
20,000	384,000.00	384,000.00	0.00
500,000	9,600,000.00	9,600,000.00	0.00
3,000,000	57,600,000.00	57,600,000.00	0.00

Observation: Both representations agree for these particular test fares because all five fares are whole-rupee values. These values are represented exactly by double at the tested totals.

This does not mean that double is always exact for monetary calculations.

4. Integer Overflow Analysis

The maximum value of a typical 32-bit signed int is:

$2,147,483,647$

The total in paise for 3,000,000 journeys is:

$$57,600,000 \times 100 = 5,760,000,000 paise$$

Since:

$$5,760,000,000 > 2,147,483,647$$

A 32-bit signed int cannot store the network total in paise. Signed integer overflow causes undefined behaviour in C++.

Therefore, long long is the appropriate choice for accumulating large network totals.

5. Compiler Warnings – -Wall -Wextra

The program is compiled using:

g++ -std=c++17 -Wall -Wextra task10.cpp -o task10

These flags enable additional compiler warnings.

Warning category	**Meaning**
Unused variable	A declared variable is never used.
Unused parameter	A function parameter is not used.
Uninitialized variable	A variable may be used before receiving a value.

Signed/unsigned comparison	Different integer types are compared.
Conversion warning	A value may lose information when converted to another type.

Evaluation: Compiler warnings help identify possible programming errors before execution. The actual warnings depend on the source code and compiler version. A clean compilation with no warnings is preferable.

6. Reasons for Differences

Differences between expected and actual results may occur due to:

1. Floating-point representation: some decimal fractions cannot be represented exactly in binary.
2. Integer overflow: a 32-bit int cannot store sufficiently large paise totals.
3. Incorrect type conversion: converting a floating-point value into an integer may discard fractional information.
4. Rounding differences: repeated calculations may produce slightly different results if rounding is performed at different stages.

For example: $0.1 + 0.2 \neq 0.3$ in exact binary floating-point arithmetic, even though the displayed result may look like 0.30.

7. Alternative Concept – Comparing Data Types

If the network total were accumulated using different data types:

Data type	**At 3 million journeys**	**Main limitation**

int	Overflows in paise	Limited range
long long	Stores 5.76 billion paise safely	Very large but finite range
double	Stores the total approximately or exactly for these particular values	Some decimal fractions are inexact

Time complexity:

All three types perform one addition per journey.

$T(n) = O(n)$

Space complexity:

Each accumulator requires constant memory.

$S(n) = O(1)$

Final evaluation: long long using integer paise is the most reliable choice for the metro's financial calculations because it avoids floating-point representation errors and supports large totals.

8. Conclusion

The comparison demonstrates that double and integer-paise totals agree for the selected test data. However, a 32-bit int cannot safely store the total paise for 3 million journeys. Therefore, integer-paise calculations using long long provide a more dependable solution for large-scale metro transactions.